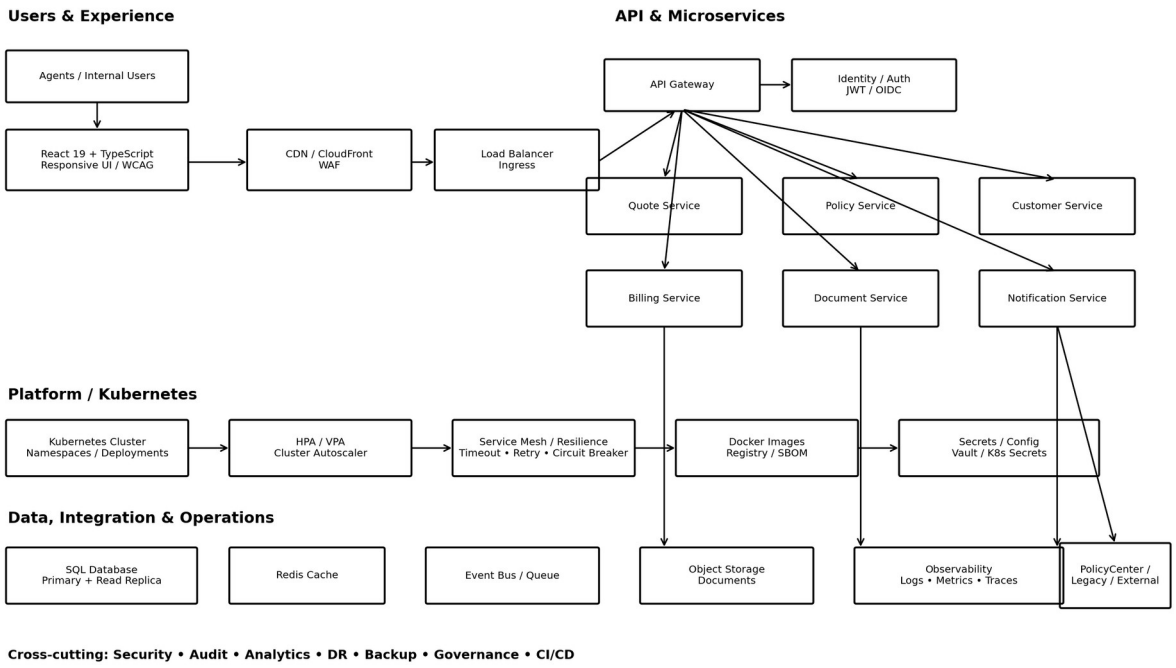
GRANGE PERSONAL LINES

END-TO-END ENTERPRISE SOLUTION ARCHITECTURE

UI + APIs + Microservices + API Gateway + Cloud + Docker + Kubernetes + Scalability + Resilience + Security + Data + Observability + Analytics + DevSecOps



Target-state architecture and Solution Architect operating model

1. Architecture Vision

This document expands the supplied Grange Personal Lines Agent Portal architecture into an end-to-end enterprise insurance solution. The supplied material establishes the portal scope, React technology stack, PolicyCenter/legacy integrations, Agency Access Management, AWS/API Gateway/Lambda/DynamoDB components, F5/load balancing, observability, accessibility and CI/CD quality practices. The additional cloud-native, microservice, Kubernetes, resilience, security, database and operational patterns in this document are recommended target-state architecture patterns for a complete enterprise solution.

Source-derived architecture

- Unified Agent Portal covering dashboard, accounts, quotes, policies, transactions, Auto, Home and Umbrella.
- React 19 + TypeScript with Zustand, TanStack Router/React Query, Axios, React-Bootstrap, Jest and RTL.
- Integration with New PL PolicyCenter, CL PolicyCenter, Legacy PL, ESB/Integration Gateway and external services.
- Agency Access Management for authentication/token management.
- F5 Load Balancer, AWS API Gateway/Lambda/DynamoDB and cloud delivery components are shown in the supplied architecture.
- NFRs include approximately 2-second response target, WCAG 2.1 AA, PII masking for analytics, graceful integration failure handling, ELK logging and minimum 80% unit coverage.

Recommended enterprise target-state additions

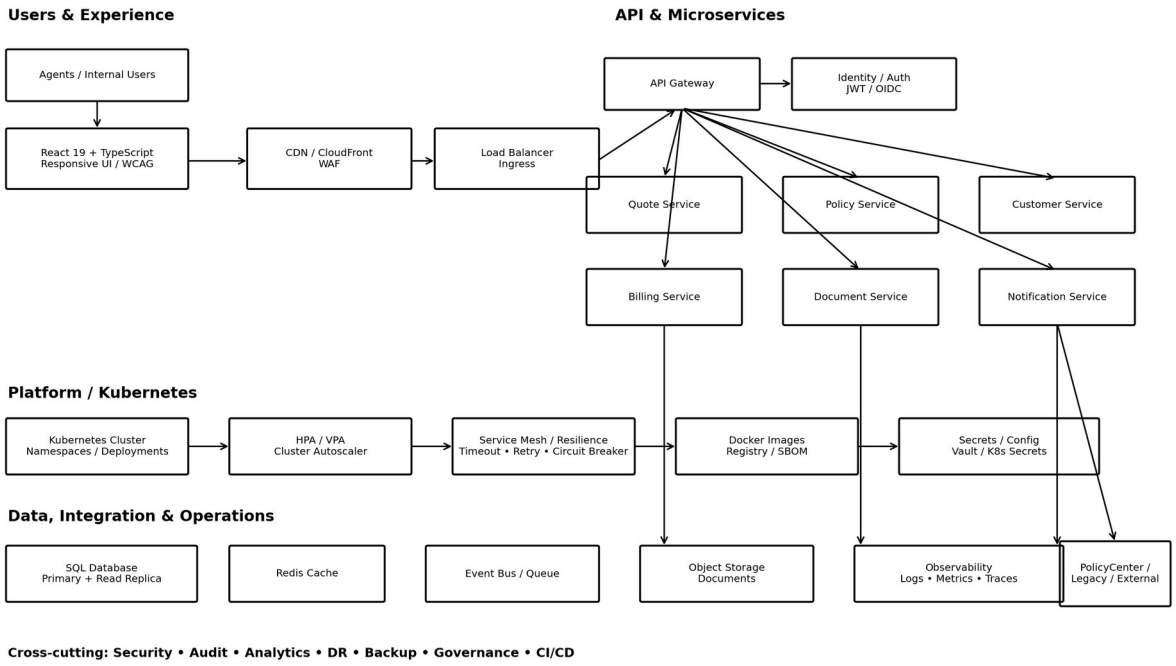
- Domain-aligned microservices with clear ownership and API contracts.
- Kubernetes orchestration for stateless services, HPA/VPA and cluster autoscaling.
- Docker image standardization, private registry, vulnerability scanning and SBOM.
- API Gateway policies for authentication, rate limiting, throttling, routing, versioning and observability.
- Resilience patterns: timeout, retry with backoff, circuit breaker, bulkhead, idempotency and fallback.
- Primary/read-replica data strategy, Redis caching, event-driven processing and object storage.
- Centralized metrics, logs and distributed tracing with SLO-driven alerting.

2. End-to-End Business Capability Map

Domain	Capabilities	Primary services
Identity	Login, token, user/agency context, authorization	Identity / Access Service
Dashboard	Open quotes, policy changes, cancellations, recently viewed, alerts	Dashboard / Aggregation Service
Customer	Account search, customer details, contact information	Customer Service
Quote	Auto, Home, Umbrella quote lifecycle	Quote Service
Risk	Driver, vehicle, property, risk enrichment	Risk / Rating Integration
Policy	Create, issue, amend, cancel, reinstate, rewrite	Policy Service
Billing	Bill plan, payment, billing history	Billing / Payment Service
Claims	Claim view and FNOL redirection	Claims Integration Service
Documents	View/upload policy documents	Document Service
Notification	Email/SMS/in-app notifications	Notification Service
Analytics	Usage, funnel, performance and business analytics	Analytics Platform

3. C4 Context / Container View

The following target architecture separates the user experience, edge layer, API/microservice layer, platform layer, data layer and enterprise integrations.



Recommended target-state diagram. It is an architectural expansion of the supplied Grange architecture rather than a claim that every component shown is already deployed.

4. UI / Front-End Architecture

4.1 UI layers

Layer	Design
Presentation	React 19 + TypeScript, responsive layout, accessibility-first components.
Routing	TanStack Router with route-level guards and lazy loading.
State	Zustand for client workflow state; TanStack Query for server state/cache.
Forms	React Hook Form + schema validation for complex insurance forms.
Design system	Reusable Grange core theme/components; consistent controls and accessibility.
API client	Axios/fetch wrapper with correlation ID, timeout, error mapping and token handling.
Telemetry	Analytics events, performance metrics and error telemetry with PII controls.

4.2 UI module structure

- Shell/Header/Navigation
- Dashboard
- Global Search
- Account
- Quote
- Policy
- Billing
- Claims
- Documents
- Notifications
- Admin/Reference Data
- Common Form Components
- Error/Empty/Loading states

4.3 UI standards

- No business rule should be duplicated unnecessarily in the UI if the authoritative rule belongs to a backend service.
- Every API call must have loading, success, validation, timeout and failure behavior.
- Use optimistic UI only where business consistency permits it.
- Mask PII in analytics events and never send payment secrets or sensitive customer fields to analytics.
- Use route guards and capability checks for Internal User restrictions.
- Support keyboard navigation, screen readers, focus management, color contrast and responsive breakpoints.

5. API Architecture

5.1 API Gateway responsibilities

- TLS termination / secure ingress
- Authentication and token validation
- Authorization / scopes
- Routing to microservices
- Rate limiting and throttling
- Request/response validation
- API versioning
- Correlation ID propagation
- CORS policy
- Timeout enforcement
- Observability
- WAF integration
- Canary / weighted routing where required

5.2 API standards

Pattern	Recommendation
Versioning	/api/v1/... with backward compatibility policy.
Idempotency	Required for purchase, payment and other retryable state-changing operations.
Errors	Standard error envelope with code, message, correlationId and validation details.
Timeouts	Set per dependency; never allow unbounded waits.
Pagination	Cursor or page-based pagination for account/quote/policy lists.
Correlation	X-Correlation-ID propagated end-to-end.
Security	JWT/OIDC validation, scopes/claims and least privilege.
Rate limits	Per client/user/agency and critical endpoint.

5.3 Example API surface

Endpoint	Purpose
POST /api/v1/quotes	Create quote
GET /api/v1/quotes/{id}	Get quote
PUT /api/v1/quotes/{id}/drivers	Update drivers
PUT /api/v1/quotes/{id}/vehicles	Update vehicles
PUT /api/v1/quotes/{id}/coverages	Update coverages
POST /api/v1/quotes/{id}/rate	Request rating
POST /api/v1/quotes/{id}/purchase	Purchase/issue
GET /api/v1/accounts/{id}	Get account
GET /api/v1/policies/{id}	Get policy
GET /api/v1/policies/{id}/billing	Get billing
POST /api/v1/payments	Create payment

6. Microservices Architecture

The recommended service boundaries are business-capability oriented rather than UI-screen oriented. This reduces coupling and allows independent scaling and deployment.

Service	Responsibilities	Data ownership
Customer Service	Customer/account profile, contacts, agency relationships	Customer DB
Quote Service	Quote lifecycle, quote state, orchestration	Quote DB
Rating Service	Rating requests and responses; integrates with rating engines	Rating DB/cache
Policy Service	Policy lifecycle and PolicyCenter integration	Policy DB / system-of-record integration
Billing Service	Bill plan, invoices, payment orchestration	Billing DB
Payment Service	Payment authorization/tokenized payment integration	Payment metadata only; no raw card data
Document Service	Document metadata, secure object storage access	Document DB + object storage
Claims Service	Claim lookup/FNOL integration	Claims integration data
Notification Service	Email/SMS/in-app events	Notification DB/queue
Search Service	Search/indexing across allowed entities	Search index
Reference Data Service	Products, states, coverage defaults, lookup values	Reference DB/cache

Service design rules

- Own data; do not share database tables across services.
- Prefer synchronous APIs for immediate user decisions and asynchronous events for long-running/non-critical work.
- Keep services stateless where possible.
- Use contract tests for service APIs/events.
- Define SLO, ownership, runbook and escalation path for every production service.

7. Resilience & Error Handling

7.1 Timeout

Every outbound dependency call should have an explicit timeout based on business expectations. A timeout should produce a controlled error response, log the dependency and correlation ID, and avoid tying up application threads.

7.2 Retry

- Retry only transient failures.
- Use exponential backoff with jitter.
- Cap retry count and total retry duration.
- Do not blindly retry payment, purchase or non-idempotent operations.
- Propagate an idempotency key for safe retries.

7.3 Circuit breaker

State	Behavior
Closed	Calls flow normally; failures are counted.
Open	Calls are rejected immediately for a cool-down period; fallback/meaningful message is returned.
Half-open	Limited trial calls determine whether dependency has recovered.

7.4 Bulkhead

Separate thread pools/concurrency limits or workload queues for high-risk dependencies so one failing integration cannot consume all service capacity.

7.5 Error classification

Class	Example	Response
Validation	Missing DOB / invalid address	HTTP 400 with field-level errors
Authentication	Invalid/expired token	401; refresh/re-authenticate
Authorization	Insufficient permission	403
Not found	Account/quote does not exist	404
Business rule	Ineligible product	409/422 with business code
Dependency timeout	PolicyCenter unavailable	504 or controlled domain error
Rate limited	Too many calls	429 with retry guidance
Unexpected	Unhandled service error	500; hide internals; correlation ID

8. Load Balancing & Scalability

8.1 Load balancing

- Internet/edge traffic → CDN/WAF → external load balancer/ingress.
- Ingress → Kubernetes services.
- Service-to-pod distribution through Kubernetes Service.
- Use readiness probes so unhealthy pods stop receiving traffic.
- Use liveness probes for deadlock/recovery cases.
- Use topology-aware distribution and pod anti-affinity for resilience.

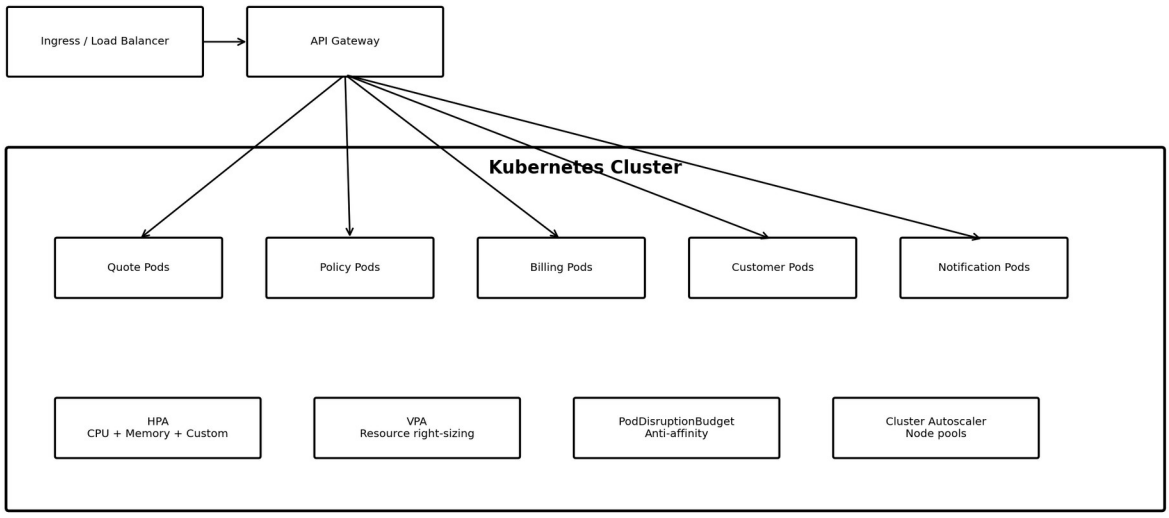
8.2 Horizontal scaling

Horizontal scaling increases the number of service instances/pods. It is the preferred pattern for stateless APIs.

Trigger	Example action
CPU	Scale when sustained CPU crosses configured threshold.
Memory	Scale when memory utilization indicates pressure.
RPS	Scale based on requests per second.
Latency	Scale when p95/p99 latency breaches threshold.
Queue depth	Scale consumers based on event backlog.

8.3 Vertical scaling

Vertical scaling increases CPU/memory allocated to a pod or node. Use it when the workload cannot efficiently scale horizontally or for database/platform sizing. Kubernetes VPA can recommend or apply resource changes depending on operating mode.



9. Docker & Kubernetes Architecture

9.1 Docker

- Use minimal base images.
- Run as non-root.
- Pin dependency versions and scan images.
- Generate SBOM.
- Do not place secrets in images.
- Use multi-stage builds for front end and services.
- Tag immutable release images.

9.2 Kubernetes objects

Object	Purpose
Namespace	Environment/team isolation
Deployment	Desired replicas and rolling updates
Service	Stable internal service endpoint
Ingress	HTTP routing into cluster
ConfigMap	Non-secret configuration
Secret / external secret	Sensitive configuration reference
HPA	Horizontal pod scaling
VPA	Vertical resource recommendation/scaling
PDB	Maintain availability during disruptions
NetworkPolicy	Restrict pod-to-pod traffic
ServiceAccount / RBAC	Least-privilege workload identity

9.3 Kubernetes production controls

- Readiness/liveness/startup probes
- Resource requests/limits
- Pod anti-affinity
- Multiple availability zones where supported
- Rolling deployments
- Pod disruption budgets
- Network policies
- RBAC
- Secrets management
- Admission/image policies
- Audit logging
- Cluster/node monitoring

10. Cloud Architecture

The supplied architecture uses AWS components such as CloudFront, WAF, API Gateway, Lambda, DynamoDB, Secrets Manager and S3, with AWS region/account boundaries shown in the technical architecture deck. The following is a recommended enterprise cloud layout.

Layer	AWS-oriented target
Edge	Route 53 / DNS → CloudFront → WAF
Web	S3 static React assets or managed web hosting
API	API Gateway → private integration / load balancer
Compute	EKS for microservices; Lambda for lightweight event/utility functions
Data	RDS/Aurora or equivalent relational store; DynamoDB where key-value/serverless access fits
Cache	ElastiCache/Redis
Messaging	SQS/SNS/EventBridge/Kafka-equivalent
Files	S3 with encryption and lifecycle policies
Secrets	Secrets Manager / KMS
Observability	CloudWatch + centralized logs/traces/metrics; AppDynamics/ELK as required by project

Architecture selection should follow Grange enterprise standards and approved cloud landing-zone patterns. This document does not assert that every recommended AWS service is currently deployed.

11. Database & Data Architecture

11.1 Data ownership

- Each microservice owns its schema/data.
- Avoid distributed transactions across services where possible.
- Use APIs/events rather than direct table access.
- Define retention, archival and deletion policies by data class.

11.2 Relational data

Insurance transaction data such as customer/account, quote, policy, billing and audit records typically benefits from relational consistency. Use normalized transactional schemas with indexes based on access patterns and read replicas for read-heavy workloads.

11.3 Caching

- Cache reference data, product metadata and safe read-heavy data.
- Never cache sensitive data without explicit security/TTL design.
- Define cache TTL and invalidation rules.
- Protect against cache stampede with request coalescing or jitter.

11.4 Database security

- Encryption at rest using managed keys.
- TLS for database connections.
- Private network placement; no public database endpoints.
- Least-privilege database identities.
- Secrets in a managed secret store, not source code.
- Audit access and privileged operations.
- Parameterized queries/ORM protections.
- Backup, restore and point-in-time recovery.
- Data masking/tokenization in non-production environments.
- Separate production and non-production credentials/accounts.

12. Enterprise Security Architecture

Control	Design
Identity	Agency Access Management / OIDC/OAuth-style token model; centralized authentication.
Authorization	Claims/scopes/capabilities; server-side authorization on every protected API.
Secrets	Managed secret store; rotation; no secrets in source or Docker images.
Encryption	TLS in transit; encryption at rest; managed key service.
Network	Private subnets for workloads/data; WAF and security groups/network policies.
Application security	Input validation, output encoding, dependency scanning, SAST/DAST, API security testing.
Container security	Image scanning, signed/approved images, non-root, minimal images.
Kubernetes security	RBAC, NetworkPolicy, service identities, admission controls.
PII	Classification, masking, minimum collection, controlled analytics, non-prod masking.
Audit	Security and business audit events with retention controls.

Threats to explicitly model

- Broken access control
- Credential/token theft
- Injection
- Sensitive data exposure
- API abuse/rate attacks
- Malicious container/image
- Lateral movement in cluster
- Misconfigured storage
- Insider access
- Supply-chain vulnerabilities

13. Monitoring, Logging & Observability

The supplied Grange material calls out AppDynamics and ELK/centralized logging. The recommended target is unified metrics, logs and traces across browser, gateway, services, database and external dependencies.

Signal	Examples	Key fields
Metrics	RPS, error rate, CPU, memory, saturation, queue depth	service, env, region
Logs	Application, security, audit, dependency errors	timestamp, correlationId, user/agency pseudonymous ID
Traces	UI → gateway → service → PolicyCenter	traceId, spanId, dependency
Business	Quote conversion, issue rate, payment success	product, state, channel

Recommended SLOs

- Availability SLO per critical service
- p95/p99 latency SLO
- Error-rate SLO
- Quote completion / issue success SLO
- Payment success SLO
- Dependency availability/error budget

Alert examples

- 5xx spike
- p99 latency breach
- Circuit breaker open
- Database connection pool saturation
- Queue backlog growth
- Pod crash loop
- Node capacity pressure
- Certificate expiry
- Authentication failure spike
- Payment failure spike

14. Analytics & Business Intelligence

The source requires Google Analytics tagging and PII masking. Extend this into an analytics architecture that separates product-usage telemetry from sensitive insurance/customer data.

Analytics layer	Events
Experience analytics	Login, search, quote start, step completion, quote display, purchase, payment outcome
Operational analytics	API latency, error, dependency availability, retries, circuit breaker state
Business analytics	Quote-to-bind conversion, product mix, funnel abandonment, time-to-quote
Quality analytics	Accessibility defects, client errors, test coverage, deployment quality

PII rules

- Do not send names, email, phone, address, DOB, payment details or raw policy/customer identifiers to general web analytics.
- Use approved pseudonymous IDs where needed.
- Mask PII using the source-defined mechanism and enforce it through automated tests/review.
- Restrict raw operational logs containing PII and define retention.

15. Event-Driven Architecture

Use events where work does not need to block the agent journey.

Event	Producer	Consumers
QuoteCreated	Quote Service	Analytics, Notification, Audit
QuoteRated	Rating/Quote Service	UI status, Analytics
PolicyIssued	Policy Service	Document, Billing, Notification, Analytics
PaymentCompleted	Payment Service	Billing, Policy, Notification
DocumentGenerated	Document Service	Notification, Audit
ClaimUpdated	Claims Integration	Dashboard, Notification

Event rules

- Events are immutable facts, not commands.
- Include eventId, eventType, version, timestamp, correlationId and aggregate ID.
- Consumers must be idempotent.
- Use dead-letter queues for poison messages.
- Monitor lag and retry counts.
- Version event schemas compatibly.

16. CI/CD & DevSecOps

The supplied material shows GitLab repositories, develop/release/main branches and CI/CD stages including prepare, lint, audit, test, build and deploy. The recommended enterprise pipeline adds security, image and infrastructure controls.

Stage	Controls
Source	Branch/MR policy, code owners, secret detection
Build	Dependency install, reproducible build, artifact generation
Lint	ESLint / formatting
Unit test	Jest/RTL; minimum 80% coverage
SAST	Static code analysis
SCA	Dependency vulnerability scan
Container	Docker build, image scan, SBOM, signing
IaC	Terraform lint/scan/plan
DAST	API/web security testing in appropriate environment
Deploy	Automated promotion with approvals for production
Post-deploy	Smoke tests, health checks, rollback/canary verification

17. Environment & Deployment Strategy

Environment	Purpose	Typical controls
DEV	Developer integration	Fast feedback; synthetic/test data
QA/SIT	System/integration testing	Stable interfaces; automated regression
UAT	Business validation	Production-like configuration and controlled data
PERF	Performance/capacity	Representative workload
SEC	Security testing	DAST/penetration/security validation
PROD	Live business	Change approval, monitoring, DR, controlled access

Deployment patterns

- Rolling deployment
- Blue/green for high-risk releases
- Canary for selected services
- Feature flags for controlled business rollout
- Automated rollback on failed health/SLO checks

18. High Availability, Backup & Disaster Recovery

Area	Target approach
Application	Stateless services across multiple nodes/zones.
Kubernetes	Multiple worker nodes, anti-affinity, PDB and autoscaling.
Database	Multi-AZ/managed HA, backups and point-in-time recovery.
Files	Object storage replication/versioning/lifecycle.
Messaging	Durable queues/events and replay strategy.
Secrets	Replicated/managed secret store with controlled recovery.
DR	Documented recovery region/account strategy aligned with business RTO/RPO.

DR validation

- Define RTO/RPO per business capability.
- Run backup restore tests.
- Run dependency-failure tests.
- Run failover/failback exercises.
- Document recovery runbooks.
- Measure actual recovery time rather than assuming it.

19. Performance Engineering

- Baseline API latency and browser performance.
- Define p50/p95/p99 targets.
- Load test normal and peak quote volumes.
- Test downstream dependency latency.
- Test database query/index behavior.
- Use caching where safe.
- Use pagination and payload shaping.
- Avoid N+1 service calls.
- Use asynchronous processing for non-blocking work.
- Capacity-test HPA and cluster scaling behavior.

Performance test scenarios

Scenario	What to validate
Normal load	Expected users/transactions with target latency
Peak load	Peak agency/quote volume
Spike	Sudden traffic increase
Soak	Long-duration stability
Dependency slow	Policy/rating service latency
Dependency down	Circuit breaker/fallback behavior
Database pressure	Connection pool, CPU, I/O and query latency

20. Testing Strategy

Level	Scope
Unit	Components, business rules, service classes
Component	Microservice API behavior with mocks/contract dependencies
Contract	Provider/consumer API and event compatibility
Integration	PolicyCenter/legacy/external integrations
E2E	Agent journeys: login→dashboard→quote→purchase→payment
Performance	Load, stress, spike, soak
Security	SAST, SCA, DAST, penetration and authorization tests
Accessibility	WCAG 2.1 AA validation
Resilience	Timeout, retry, circuit breaker, dependency failure
DR	Restore/failover tests

21. Solution Architect Governance Model

Lifecycle	SA ownership
Discovery	Current-state, scope, assumptions, constraints
Requirements	Technical requirements/NFRs and dependencies
Architecture	SAD/HLD, C4, integration and security architecture
Design	API/data/component design review
Development	Architecture compliance, technical guidance, critical reviews
Testing	NFR/performance/security/resilience acceptance
Release	Production readiness, rollback and risk acceptance
Production	RCA, capacity, technical debt, architecture evolution

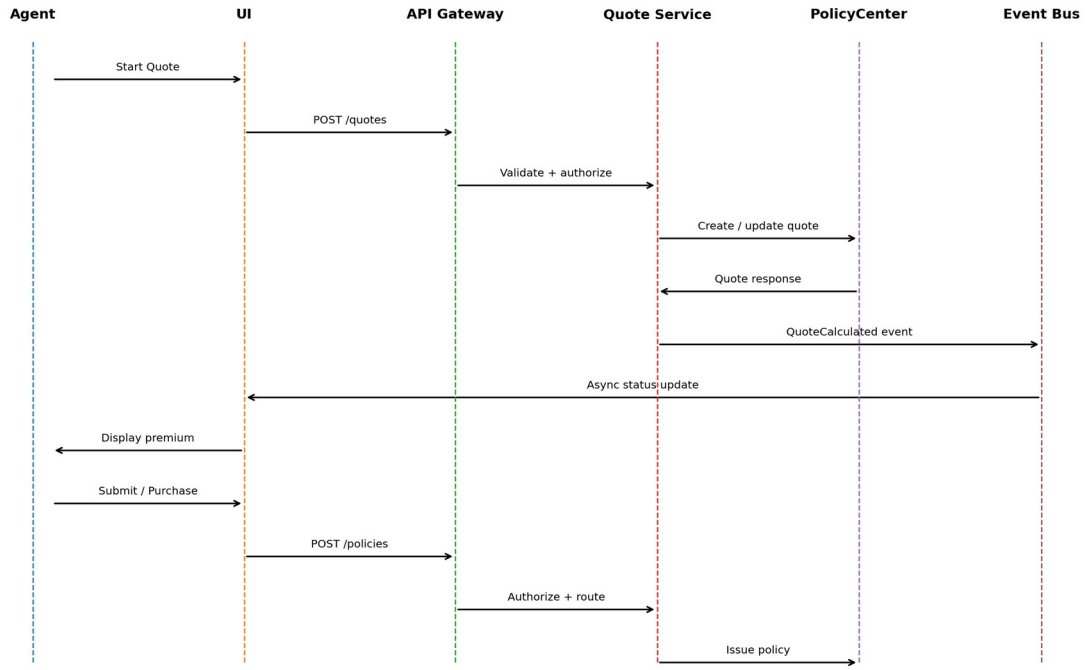
Architecture review gates

- Gate 1 – Scope & NFR approval
- Gate 2 – Architecture approval
- Gate 3 – Detailed design/API contract approval
- Gate 4 – Security & performance readiness
- Gate 5 – Production readiness
- Gate 6 – Post-production architecture review

22. RACI – End-to-End Delivery

Activity	SA	PM	BA	Tech Lead	Dev	QA	Security	DevOps
Scope	A/R	A	R	C	I	I	I	I
NFR	A/R	C	C	R	C	C	C	C
Architecture	A/R	I	C	R	C	C	C	C
API design	A	I	C	R	R	C	C	C
Security architecture	A	I	I	C	C	I	R	C
Implementation	A	I	I	R	R	I	I	C
Testing	A	C	C	C	C	R	C	C
Deployment	A	C	I	C	C	C	C	R
Production readiness	A	C	I	R	C	R	R	R
Architecture governance	A/R	C	I	R	C	C	C	C

23. Detailed Quote-to-Policy Flow



1. Agent logs in; access token is obtained/validated through Agency Access Management.
2. Dashboard loads through API Gateway and aggregation calls.
3. Agent starts Auto/Home/Umbrella quote.
4. Quote Service validates input and creates quote context.
5. Enrichment services retrieve risk/customer data through controlled integrations.
6. Rating request is sent to rating/PolicyCenter dependencies.
7. Long-running or non-blocking events are published to the event bus.
8. Quote premium and eligibility are returned to the UI.
9. Agent confirms coverages and submits/purchases.
10. Policy Service orchestrates issue flow with PolicyCenter.
11. Payment Service handles payment integration using tokenized payment mechanisms.
12. Policy issued event triggers documents, billing updates, notification and analytics.

24. Failure Scenarios & Recovery

Failure	Expected behavior
API Gateway unavailable	Traffic fails fast; edge health/monitoring alerts; no cascading retries.
Quote Service pod down	Kubernetes routes to healthy replica; HPA/PDB maintain availability.
PolicyCenter slow	Timeout + circuit breaker; preserve quote context where business permits; show meaningful message.
PolicyCenter unavailable	Open circuit; prevent repeated calls; allow safe retry/recovery workflow.
Database primary issue	Failover according to managed DB design; service health check; alert.
Redis unavailable	Bypass cache and retrieve from source if safe; avoid making cache a single point of failure.
Event consumer failure	Retry, then DLQ; alert on lag; replay after fix.
Payment timeout	Do not blindly retry purchase; use idempotency and payment-status reconciliation.
External enrichment unavailable	Use fallback/manual path only if business rules allow; otherwise return clear eligibility/incomplete-data message.
Bad deployment	Automated health check triggers rollback/canary stop.

25. Cost Optimization

- Right-size Kubernetes requests/limits using observed usage.
- Use HPA and cluster autoscaler to avoid idle capacity.
- Use caching for safe high-read workloads.
- Archive old documents/logs to lower-cost storage tiers.
- Set log retention by business/security requirement.
- Use asynchronous workloads where appropriate.
- Track cost per service and environment.
- Review unused load balancers, volumes, snapshots and public resources.

26. Mandatory Architecture Artifacts

Artifact	Owner	When
Architecture Vision	SA	Discovery
Current-State Architecture	SA	Discovery
Solution Architecture Document	SA	Architecture
C4 Context/Container/Component	SA	Architecture/Design
API Standards & Contracts	SA/Tech Lead	Design
Data Architecture	SA/DBA	Design
Security Architecture	SA/Security	Design
Threat Model	SA/Security	Design
ADR Register	SA	Throughout
NFR & SLO Document	SA	Requirements
Resilience Strategy	SA	Design
Performance Strategy	SA	Design/Test
Deployment Architecture	SA/DevOps	Release
DR/BCP Plan	SA/Operations	Release
Production Readiness Checklist	SA	Go-live
Runbook	Operations/SA	Go-live
RCA / Architecture Review	SA	Production

27. End-to-End Solution Architect Checklist

- Business scope and personas confirmed.
- UI journeys mapped to business capabilities.
- NFRs and SLOs documented.
- C4 architecture approved.
- Microservice boundaries and ownership defined.
- API Gateway standards defined.
- API contracts versioned.
- Authentication and authorization approved.
- Docker build and registry controls defined.
- Kubernetes topology and resource strategy defined.
- Horizontal and vertical scaling configured/tested.
- Load balancing and health probes configured.
- Timeout, retry, circuit breaker and bulkhead patterns defined.
- Database ownership, indexing, backup and security defined.
- Caching strategy defined.
- Event-driven flows and DLQ strategy defined.
- Logging, metrics and distributed tracing implemented.
- Analytics events and PII masking approved.
- SAST/SCA/DAST/container/IaC security controls integrated.
- CI/CD gates implemented.
- Performance, security, resilience and E2E tests passed.
- DR/RTO/RPO validated.
- Production readiness and rollback approved.
- Operational handover and runbooks complete.
- Architecture debt and improvement backlog created.

28. Executive Architecture Summary

The target architecture positions the Grange Personal Lines Agent Portal as a secure, resilient, observable and scalable digital insurance platform. React provides the agent experience; an API Gateway provides controlled entry and policy enforcement; business-aligned microservices isolate domains; Kubernetes provides orchestration and horizontal scaling; Docker standardizes packaging; event-driven integration handles asynchronous work; databases and caches support transactional and read-heavy workloads; PolicyCenter, legacy and external systems remain protected behind integration boundaries; and security, analytics, observability, CI/CD and disaster recovery operate as cross-cutting concerns.

The Solution Architect owns the coherence of the whole system: business journey, technical boundaries, NFRs, security, resilience, data, deployment and operational behavior must all align before the solution is considered production-ready.

Important source boundary: the supplied project files establish the Grange portal scope and several existing architecture components. Kubernetes, Docker, detailed microservice decomposition, circuit-breaker policies, HPA/VPA design, database hardening and several cloud-native controls above are recommended target-state additions requested for this end-to-end architecture; they should be validated against Grange enterprise standards before implementation.